

P2505R3 Monadic Functions for

`std::expected`

Authors: Jeff Garland

Audience: LEWG, LWG

Project: ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Contact: jeff@crystalclearsoftware.com

Introduction

With the final plenary vote of [P0798 Monadic Functions for `std::optional`](#) complete, we now have a design inconsistency with `std::expected`. [P0323 `std::expected`](#) has now also been voted into the working draft for C++23. This proposal corrects the inconsistency by adding 4 functions to `std::expected` and is targeted at C++23. The author believes this should be treated as a consistency/bug fix still in scope for C++23.

The intent from P0798 clearly states that `std::expected` should gain the same functions:

There is a proposal for `std::expected` which would benefit from many of these same ideas. If the idea to add monadic interfaces to standard library classes on a case-by-case basis is chosen rather than a unified non-member function interface, then compatibility between this proposal and the `std::expected` one should be maintained

Motivation and Scope

The following 3 functions are added to `std::optional`, but are currently not part of `std::expected`.

- `and_then` compose a chain of functions returning an `expected`
- `or_else` returns if it has a value, otherwise it calls a function with the error type
- `transform` apply a function to change the value (and possibly the type)

After feedback on the R0, this draft also proposes the addition of a fourth function for `expected` to facilitate additional cases:

- `transform_error` apply a function to change the value, otherwise call a function with error type

For example, given the following:

```

using time_expected = expected<boost::posix_time::ptime, std::string>;

time_expected from_iso_str( std::string time )
{
    try {
        ptime t = boost::posix_time::from_iso_string( time );
        return t;
    }
    catch( std::exception& e )
    {
        return unexpected( e.what() + " str was: "s + time);
    }
}

// for use with transform
ptime next_day( boost::posix_time::ptime t )
{
    return t + boost::gregorian::days(1);
}

// for use with or_else
void print_error( std::string error )
{
    cout << error << endl;
}

//valid iso string
const std::string ts ( "20210726T000000" );

```

Before the change we'd write this:

Before

```
time_expected d = from_iso_str( ts );
if (d)
{
    ptime t = next_day( *d );
}
else
{
    print_error( d.error() );
}
```

And after, this:

After

```
// chain a series of functions until there's an error
auto d = from_iso_str ( ts )
        .or_else( print_error )
        .transform( next_day );
```

Jonathan Wakely has a live demo illustrating the api here <https://godbolt.org/z/3a1j6d63a>. For more examples see: <https://godbolt.org/z/K7rbhx9oo>.

Design Considerations

`expected<void, E>`

Compared with `optional`, `expected` provides the capability for `void` as a function return type. This complicates the design compared to `optional`. As currently specified the `void` specialization of `expected`, `expected<void, T>` removes the `value_or` function since `void` cannot be initialized to anything else.

While the primary implementation for this paper excludes `transform` for `expected<void, E>` and the original version removed this overload, feedback from reviewers suggested the overload is valuable. Thus the new version allows for this overload.

`transform` to `expected<void, E>` return

Consider the following code.

```
struct E{};
auto x = expected<int, E>(4).transform([](int){});
```

The original specifications and implementations would fail to compile this code. A reasonable alternative is to return `expected<void, E>`. The specification has been changed to allow this behavior.

`transform_error` function

The R0 version of this paper did not include this function to match optional, but several reviewers noted the important utility of this case. Note that in the reference implementation of this paper, the function is called `map_error`. The R1 version of the paper names the function `transform_error` after discussions with various reviewers.

Changing of error return types

One complication of the monadic functions for `expected` as compared to `optional` is the ability to change the return error return type on subsequent function calls. Optional, of course, can only return `std::nullopt_t`.

Thus we would like to support use cases like the following:

```
std::expected<SomeUserType, InternalError> initialize();

std::expected<SomeUserType, UserError>
makeUserError(InternalError err)
{
    return std::unexpected( UserError(err) );
};
```

```
std::expected<SomeUserType, UserError> result = initialize().or_else( m
```

Note that the changing of the error type applies to the `or_else` and `transform_error` overloads only.

Use of deducing-this in the specification

This question has been asked several times including in the LEWG 2022-05-03 meeting. While less concise, the author prefers to keep the specification consistent with the rest of the c++23 library and especially `std::optional`.

Limited access to compilers that implement deducing-this make it difficult to confirm correct behavior of a deducing-this based specification. In addition, the current specification does not preclude changing to a deducing-this based consistently with other library facilities in a future release.

Adding `error_or` functions

It has been suggested that `error_or` functions be added to the proposal. While the author is sympathetic to the suggestion, since consistency with `optional` for c++23 is the intent these are not proposed. These can be added later or if LEWG would prefer the author can investigate adding them in this cycle.

Free versus member functions

One design question that has been raised is the use of member functions instead of free functions. The author is sympathetic to this view, but believes this can only be done by changing both `optional` and `expected`. The expressed benefits of free functions would be:

- better compatibility with future library facilities
- more generic code – singular specification

While in theory, free functions provide consistency with some future version of the library where functions such as `then` and `upon_error` from P2300 `std::execution` would be part of the standard library. However, `then` is specified using concepts such as `std::execution::sender` which are not appropriate for a generalized `then` functions for `optional` and `expected`. In fact, attempting to specify this as free functions now might trample on names for future library free functions.

Since the author is unaware of any existing practice with providing these functions and the purpose of the paper is consistency with monadic optional functions already voted into the working draft, the author will not pursue this option. A fully thought out paper with existing practice would be required to make this change.

Note that the author's experience with `std::expected` implementing json parsing is that `expected` would be extremely cumbersome for the task without these functions. So shipping C++23 without these functions for an unclear future benefit would be a major mistake for users.

Other Languages

Rust provides a result type with similar functions monadic functions.

<https://doc.rust-lang.org/std/result/enum.Result.html>

Implementations

Jonathan Wakely has implemented the full proposal here

<https://github.com/jwakely/gcc/blob/expected/libstdc%2B%2B-v3/include/std/expected>.

The core of the proposal was originally implemented by Sy Brand

<https://github.com/TartanLlama/expected> with [documentation here](#).

Wording

Feature Test Macro

Update the `__cpp_lib_expected` feature test macro from P0323. Although this could be considered optional since `expected` is not yet a c++ feature in any implementations.

Class template `expected` [`expected.expected`]

After `value_or` functions add the new functions:

```
template<class U> constexpr T value_or(U&&) &&;

// [expected.monadic], monadic operations
template <class F> constexpr auto and_then(F&& f) &;
template <class F> constexpr auto and_then(F&& f) &&;
template <class F> constexpr auto and_then(F&& f) const&;
template <class F> constexpr auto and_then(F&& f) const&&;
template <class F> constexpr auto or_else(F&& f) &;
template <class F> constexpr auto or_else(F&& f) &&;
template <class F> constexpr auto or_else(F&& f) const&;
template <class F> constexpr auto or_else(F&& f) const&&;
template <class F> constexpr auto transform(F&& f) &;
template <class F> constexpr auto transform(F&& f) &&;
template <class F> constexpr auto transform(F&& f) const&;
template <class F> constexpr auto transform(F&& f) const&&;
template <class F> constexpr auto transform_error(F&& f) &;
template <class F> constexpr auto transform_error(F&& f) &&;
template <class F> constexpr auto transform_error(F&& f) const&;
template <class F> constexpr auto transform_error(F&& f) const&&;
// [expected.object.swap], swap
```

Add a new sub-clause Monadic operations for [`expected.monadic`] between [`expected.observe`] and [`expected.equality`]

1 `template <class F> constexpr auto and_then(F&& f) &;`

2 `template <class F> constexpr auto and_then(F&& f) const&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(value())>>`.

Constraints:

- `is_same_v<U::error_type, E>` is `true`,
- `is_copy_constructible_v<E>` is `true`, and
- `is_copy_constructible_v<U>` is `true`.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (has_value()) {  
    return invoke(std::forward<F>(f), value());  
}  
else {  
    return U(unexpect, error());  
}
```

1 `template <class F> constexpr auto and_then(F&& f) &&;`

2 `template <class F> constexpr auto and_then(F&& f) const&&;`

Let U be

`remove_cvref_t<invoke_result_t<F, decltype(std::move(value()))>>`.

Constraints:

- `is_same_v<U::error_type, E>` is `true`,
- `is_move_constructible_v<E>` is `true`, and
- `is_move_constructible_v<U>` is `true`.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```

if (has_value()) {
    return invoke(std::forward<F>(f), std::move(value()));
}
else {
    return U(unexpect, std::move(error()));
}

```

1 `template <class F> constexpr auto or_else(F&& f) &;`

2 `template <class F> constexpr auto or_else(F&& f) const&;`

Let G be

`remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

Constraints:

- `is_same_v<G::value_type, T>` is true,
- `is_copy_constructible_v<T>` is true, and
- `is_copy_constructible_v<G>` is true.

Mandates: G is a specialization of `expected`.

Effects: Equivalent to:

```

if (has_value()) {
    return G(value());
} else {
    return invoke(std::forward<F>(), error());
}

```

1 `template <class F> constexpr auto or_else(F&& f) &&;`

2 `template <class F> constexpr auto or_else(F&& f) const&&;`

Let G be

```
remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>.
```

Constraints:

- `is_same_v<G::value_type, T>` is `true`,
- `is_move_constructible_v<T>` is `true`, and
- `is_move_constructible_v<G>` is `true`.

Mandates: G is a specialization of `expected`.

Effects: Equivalent to:

```
if (has_value()) {  
    return std::move(*this);  
}  
else {  
    return invoke(std::forward<F>(f)(std::move(error())));  
}
```

1 `template <class F> constexpr auto transform(F&& f) &;`

2 `template <class F> constexpr auto transform(F&& f) const&;`

Let U be `remove_cvref_t<invoke_result_t<F, decltype(value())>>`.

Mandates: U is a non-array object type other than `in_place_t`, `unexpected_t`, or a specialization of `unexpected`. The declaration

```
U u(invoke(std::forward<F>(f), value()));
```

 is well-formed.

Constraints:

- `is_same_v<U::error_type, E>` is `true`,
- `is_copy_constructible_v<E>` is `true`, and
- `is_copy_constructible_v<U>` is `true`.

Effects: Equivalent to: if `is_same<U, void>` is false

```
if (has_value()) {  
    return expected<U, E>(in_place, ~invoke(std::forward<F>(f), value()  
}  
else {  
    return expected<U,E>(unexpected, error());  
}
```

otherwise

```
if (has_value()) {  
    return expected<void, E>(in_place, invoke(std::forward<F>(f), value()  
}  
else {  
    return expected<void,E>(unexpected, error());  
}
```

1 `template <class F> constexpr auto transform(F&& f) &&;`

2 `template <class F> constexpr auto transform(F&& f) const&&;`

Let `U` be

`remove_cvref_t<invoke_result_t<F, decltype(std::move(value()))>>`.

Mandates: `U` is a non-array object type other than `in_place_t`, `unexpected_t`, or a specialization of `unexpected`. The declaration

`U u(invoke(std::forward<F>(f), std::move(value())));` is well-formed.

Constraints:

- `is_same_v<U::error_type, E>` is true,
- `is_move_constructible_v<E>` is true, and
- `is_move_constructible_v<U>` is true.

Effects: Equivalent to: if `is_same<U, void>` is false

```
if (has_value()) {
    ~expected<U, E>(in_place, ~invoke(std::forward<F>(f), value()))~
}
else {
    return expected<U,E>(unexpected, error());
}
```

otherwise

```
if (has_value()) {
    return expected<void, E>(in_place, invoke(std::forward<F>(f), std::m
}
else {
    return expected<void,E>(unexpected, std::move(error()));
}
```

Effects: Equivalent to:

```
if (has_value()) {
    return expected<U,E>(invoke(std::forward<F>(f), std::move(value())))
}
else {
    return U(unexpected, std::move(error()));
}
```

1 `template <class F> constexpr auto transform_error(F&& f) &;`

2 `template <class F> constexpr auto transform_error(F&& f) const&;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints:

- `is_copy_constructible_v<T>` is `true`, and
- `is_copy_constructible_v<G>` is `true`.

Effects: Equivalent to:

```
if (has_value()) {
    return expected<T, G>();
}
else {
    return expected<T,G>(unexpected, invoke(std::forward<F>(f)(error())));
}
```

1 `template <class F> constexpr auto transform_error(F&& f) &&;`

2 `template <class F> constexpr auto transform_error(F&& f) const&&;`

Let G be

`remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

Constraints:

- `is_move_constructible_v<T>` is `true`, and
- `is_move_constructible_v<G>` is `true`.

Effects: Equivalent to:

```
if (has_value()) {
    return expected<T, G>()
}
else {
    return expected<T,G>(unexpected, invoke(std::forward<F>(f)(std::move(e
}
```

Partial specialization of `expected` for void types

[expected.void]

After ❷.❸.8.6, observers – add

```
// ❷.❸.8.7, monadic
template <class F> constexpr auto and_then(F&& f) &;
template <class F> constexpr auto and_then(F&& f) &&;
template <class F> constexpr auto and_then(F&& f) const&;
template <class F> constexpr auto and_then(F&& f) const&&;
template <class F> constexpr auto or_else(F&& f) &;
template <class F> constexpr auto or_else(F&& f) &&;
template <class F> constexpr auto or_else(F&& f) const&;
template <class F> constexpr auto or_else(F&& f) const&&;
template <class F> constexpr auto transform(F&& f) &;
template <class F> constexpr auto transform(F&& f) &&;
template <class F> constexpr auto transform(F&& f) const&;
template <class F> constexpr auto transform(F&& f) const&&;
template <class F> constexpr auto transform_error(F&& f) &;
template <class F> constexpr auto transform_error(F&& f) &&;
template <class F> constexpr auto transform_error(F&& f) const&;
template <class F> constexpr auto transform_error(F&& f) const&&;
```

Partial specialization of expected for void [expected.void.observe]

After section ❷.❸.8.5 Observers [expected.void.observe]

Add section ❷.❸.8.6 Monadic [expected.void.monadic]

1 `template <class F> constexpr auto and_then(F&& f) &;`

2 `template <class F> constexpr auto and_then(F&& f) const&;`

3 `template <class F> constexpr auto and_then(F&& f) &&;`

4 `template <class F> constexpr auto and_then(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Mandates: U is a specialization of `expected`.

Effects: Equivalent to:

```
if (has_value()) {  
    return std::move(std::forward<F>(f)());  
}  
else {  
    return U(unexpect, std::move(error()));  
}
```

1 `template <class F> constexpr expected or_else(F&& f) &;`

2 `template <class F> constexpr expected or_else(F&& f) const&;`

3 `template <class F> constexpr expected or_else(F&& f) &&;`

4 `template <class F> constexpr expected or_else(F&& f) const &&;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Mandates: G is a specialization of `expected`.

Effects: Equivalent to:

```

if (has_value()) {
    return std::move(*this);
}
else {
    return std::forward<F>(f)();
}

```

1 `template <class F> constexpr auto transform(F&& f) &;`

2 `template <class F> constexpr auto transform(F&& f) const&;`

3 `template <class F> constexpr auto transform(F&& f) &&;`

4 `template <class F> constexpr auto transform(F&& f) const&&;`

Let U be `remove_cvref_t<invoke_result_t<F>>`.

Constraints: `is_same_v<U::error_type, E>` is true. E models

`copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Effects: Equivalent to:

```

if (has_value()) {
    return expected<void, E>(std::forward<F>(f)());
}
else {
    return expected<void, E>(unexpected, std::move(error()));
}

```

1 `template <class F> constexpr auto transform_error(F&& f) &;`

2 `template <class F> constexpr auto transform_error(F&& f) const&;`

3 `template <class F> constexpr auto transform_error(F&& f) &&;`

4 `template <class F> constexpr auto transform_error(F&& f) &&;`

Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

Constraints: E models `copy_constructible` for the first two overloads and `move_constructible` for the second two overloads.

Mandates: G is a specialization of `expected`.

Effects: Equivalent to:

```
if (has_value()) {
    return std::move(*this);
}
else {
    return std::move(std::forward<F>(f)(error()));
}
```

Acknowledgements

- Thanks to Jonathan Wakely for implementation and wording feedback.
- Thanks to Tomasz Kaminski for extensive wording feedback.
- Thanks to Sy Brand for the original work on optional and expected.
- Thanks to Broneck Kozicki for early feedback.
- Thanks to Arthur O'Dwyer for design and wording feedback.
- Thanks to Barry Revzin for design and wording feedback as well as shepherding in LEWG.

Revision History

Version	Date	Changes
0	2021-12-12	Initial Draft
1	2022-02-10	added <code>transform_or</code>

Version	Date	Changes
		allow <code>transform</code> on <code>expected<void, E></code>
		updated status of P0323 to plenary approved
		improved examples
		added missing overloads for <code>or_else</code>
		<code>or_else</code> signature and wording fixes
		added design discussion of changing return types
		variety of wording fixes
2	2022-04-15	updates to reference Wakely implmentation and examples
		remove unneeded invoke calls from void overloads
		<code>expected.object.swap</code> instead of modifiers in synopsis
		change constraint wording to use <code>is_copy_constructible_v</code> form for consistency
		change Let U for move overloads to <code>decltype(std::move(...))</code>
		fix <code>transform_error</code> and <code>or_else</code> constraints to be T instead of E
		rename <code>transform_or</code> to <code>transform_error</code>
		remove the <code>is_same_v</code> constraint on <code>transform_error</code>
		change <code>transform_error</code> true case to return <code>expected<T,U></code>
		remove note: U will implicitly model either copy or move constructible...
		change all the <code>if (*this)</code> to <code>if (has_value())</code> to match latest expected wording
		remove the <code>in_place</code> tag in <code>transform</code> overloads

Version	Date	Changes
		remove void from <code>invoke_results</code> for <code>expected<void, E></code> and <code>then</code> and <code>transform</code> .
		change error returns for <code>and_then~::~transform</code> to <code>return U(unexpect, error())</code>
		change Let U to Let G for <code>transform_error</code> and <code>or_else</code> for less confusion
3	2022-06-05	Add a design discussion of why deducing-this is not utilized
		Add a discussion of free function versus member functions
		mandate <code>transform</code> return type not be <code>unexpected</code> , <code>inplace_t</code> , etc
		design discussion and wording for <code>transform</code> to <code>expected<void, E></code>
		design discussion of <code>error_or</code> functions

References

1. [P0798] Sy Brand "Monadic Functions for `std::optional`"
<https://wg21.link/P0798>
2. C++ draft [optional.nomadic] <http://eel.is/c++draft/optional.monadic>
3. [P0323] Vicente Botet, JF Bastien, Jonathan Wakely `std::expected`
<https://wg21.link/P0323>
4. Sy Brand `expected` <https://github.com/TartanLlama/expected>
5. Sy Brand `expected` docs
<https://tl.tartanllama.xyz/en/latest/api/expected.html#tl-expected>
6. Jonathan Wakely full implementation
<https://github.com/jwakely/gcc/blob/expected/libstdc%2B%2B-v3/include/std/expected>
7. Jonathan Wakely live demo <https://godbolt.org/z/3a1j6d63a>
8. Rust Result <https://doc.rust-lang.org/std/result/enum.Result.html>
9. More complete examples <https://godbolt.org/z/va5fzx11f>